

Diego Fernando Sánchez Bruno

### 2.1.3 Aplicaciones con Pila

En esta sección se puede apreciar que ya estamos en condiciones para plantear algoritmos usando el TDA Pila, observándonos de la forma como está implementado.

#### Evaluar Expresiones

Para poder evaluar expresiones tenemos que tomar en cuenta que existen dos formatos de expresión muy importantes que al principio, pueden no parecer obvios. Considere la expresión Infixa  $A+B$ . ¿Que pasaría si moviéramos el operador antes de los dos operandos? la expresión resultante sería  $+AB$ . Del mismo modo, podríamos mover el operador al final de la expresión y obtendríamos  $AB+$ . Estas expresiones se ven un poco extrañas.

Estos cambios en la posición del operador con respecto a los operandos crean dos nuevos formatos de expresión, la notación Prefija y la notación Sufija (o Postfixa). La notación Prefija requiere que todos los operadores precedan a los dos operandos sobre los que actúan. La notación Sufija, por otro lado, requiere que sus operadores aparezcan después de los operandos correspondientes. Algunos ejemplos más deberían ayudar a hacer esto un poco más claro según el siguiente gráfico:



Diego Fernando Panegras Bruno

| Expresión Infixa    | Expresión Prefija | Expresión Sufija |
|---------------------|-------------------|------------------|
| $A + B$             | $+ A B$           | $A B +$          |
| $A + B * C$         | $+ A * B C$       | $A B C * +$      |
| $(A + B) * C$       | $* + A B C$       | $A B + C *$      |
| $A + B * C + D$     | $++ A * B C D$    | $A B C * + D +$  |
| $(A + B) * (C + D)$ | $* + A B + C D$   | $A B + C D + *$  |
| $A * B + C * D$     | $+ * A B * C D$   | $A B * C D * +$  |
| $A + B + C + D$     | $+++ A B C D$     | $A B + C + D +$  |

Aplicación 1: Evaluación de expresiones PostFijas

Dada la siguiente expresión:  $2 + 3 * 5 \rightarrow 235 * + = 17$

Real EvaluarPostFija (ExpPostFija : Cadena)

INICIO

//llamar al constructor de la Pila P

Para cada  $i = 1$  hasta longitud (ExpPostFija) hacer //iniciamos recorrido

INICIO

Si ExpPostFija  $[i]$  esta en ('0', '1', '2', '3', '4', '5', '6', '7', '8', '9') //si es operando metemos Pila

Entonces

P. meter (ExpPostFija  $[i]$ )

Caso contrario // Si es un operador sacamos los dos primeros

INICIO

P. sacar (Op2) // de la Pila y realizamos la operación indicada

P. sacar (Op<sub>1</sub>) // Al final el resultado lo volvemos a meter en la pila  
 Sim - Operacion = ExpPostFija[i]  
 Z = Evalua (Op1, Sim - Operacion, Op2)  
 P. meter (Z)  
 Fin

Fin

// Al final la Pila queda con un único elemento que es el resultado de la evaluación

EvaluadorPostFija = P. cima()

Fin

Real Evalua (Op1: real; Operador: char; Op2: real)

Inicio

SI OPERADOR = '^' Entonces Evalua = exp (Op2 \* ln(Op1)) FIN SI  
 SI OPERADOR = '\*' Entonces Evalua = Op1 \* Op2 FIN SI  
 SI OPERADOR = '/' Entonces Evalua = Op1 / Op2 FIN SI  
 SI OPERADOR = '+' Entonces Evalua = Op1 + Op2 FIN SI  
 SI OPERADOR = '-' Entonces Evalua = Op1 - Op2 FIN SI

FIN

Aplicación 2. Conversión de notación infija a Postfija

Dada la siguiente expresión  $2 + 3 * 5 \rightarrow 235 * +$



Materia:

Fecha:

Cadena Infixa TO Post Fija (Infixa a Cadena) {Diego Fernando Benegas  
Inicio  
// llamar al constructor de la Pila P  
Post Fija = ""  
PARA CADA i=1 to longitud (infixa) hacer // Recorremos la expresión infix  
Inicio  
Si infix[i] esta en ('0','1','2','3','4','5','6','7','8','9') // operando a la expresión postfix  
Entonces Post Fija = Post Fija + infix[i]  
CASO CONTRARIO  
Si infix[i] esta en ('(',')','\*','/','+','-','^','<','>') // si es un operador  
Entonces  
Inicio  
Salir = Falso  
Mientras No(Salir) HACER  
Inicio  
Aux = P.cima()  
Si (P.vacia) o (Prioridadinfixa / infix[i] > Prioridad Pila (aux))  
Entonces Inicio  
P.meter (infix[i]) // se mete el operador en la pila  
Salir = Verdadero  
FIN  
CASO CONTRARIO  
Inicio // si es < o = sacar el operador de la pila  
P.sacar (aux)  
Post Fija = Post Fija + aux  
FIN  
FIN // mientras su fin  
FIN  
CASO CONTRARIO // si encontramos un Paréntesis derecho se sacan  
Si infix[i] = ')' Entonces // operador de la pila hasta encontrar un  
Repetir // Paréntesis izquierdo que se elimina  
P.sacar (aux)  
Si aux < > '(' Entonces  
Post Fija = Post Fija + aux  
Hasta que aux = '('  
FIN  
Mientras no P.vacia HACER  
Inicio  
P.sacar (aux)  
Post Fija = Post Fija + aux  
FIN  
Infixa TO Post Fija = Post Fija  
FIN



|   | 1  | 2  | 3  | 4  |
|---|----|----|----|----|
| 1 | 25 | 60 | 45 | 52 |
| 2 |    |    |    |    |
| 3 |    |    |    |    |
| 4 |    |    |    |    |

Entero Prioridad Infija (C = caracter)

Diego Fernando Banezas

INITIAL

Si  $C = \{A\}$  Entonces Probabilidad infija = 4 Fin Si

Si  $C = '*'$  Entonces PrioridadInfixa = 2 **FIN** Si

SI  $C = '/'$  Entonces PrioridadInfixa = 2 FIN SI

SI  $C = '+'$  Entonces PrioridadInfixa = 1 FIN SI

SI  $C = 1$  Entonces Prioridad infinita = 1 FIN SI

Si  $C = (C^1)$  Entonces Prioridad infija = S FIN S

FIN

Entero Prioridad pila (c % caracter)

INICIO

Si  $C = \wedge$  Entonces Prioridad pila = 3 FIN SI

SI  $C = (*)$  Entonces Prioridadpila = 2 FIN SI

Si  $C = '/'$  Entonces Prioridad pila = 2 FIN SI

Si  $C = '+'$  Entonces Prioridad  $pila = 1$  FIN SI

SI  $C = (-)$  Entonces Prioridadpila  $> 1$  FIN SI

SI  $C = 'C'$  Entonces  $Prioridad_{pila} = 0$  FIN SI

Aplicación 3. Análisis de Parentesis es una expresión  
verificar si la expresión tiene correctamente colocados los paré-  
ntesis.



Materia:

Fecha:

Diego Fernando Banegas

Booleano Parentesis OK (Expresión % Cadena)

Inicio

// Llamar Constructor de Pila P

Para cada  $i \geq 1$  hasta longitud (expresión) hacer

Inicio

Si expresión  $[i] = '('$  entonces

P. meter (expresión  $[i]$ )

Si expresión  $[i] = ')'$  entonces

P. Sacar (aux)

Fin

Parentesis OK = (P.vacia)

Fin

## 2.1.4 Implementaciones del TDA Pila

En esta sección mostraremos tres implementaciones Para el TDA Pila:

- Implementación con lista
- Implementación con vectores
- Implementación con Simulación de Memoria / Punteo

### 2.1.4.1 Implementación con lista

Clase Pila

Atributos

$L$  : Lista

Metodos

Crear()

booleano vacia()

Meter (E: Tipo Elemento)

Sacar (Es E: Tipo Elemento)

Tipo elemento cima()

Fin

Constructor Pila. crear inicio

// crear Objeto L

Fin

Pila. meter (E: Tipo Elemento)

inicio

1. Inserta (1. Primero(), E)

Fin

Pila. Sacar (es E: Tipo elemento)



Materia:

Fecha:

inicio

1. recupera (1. primero(), E)

1.0. Suprime (1. primero)

Fin

Diego Fernando Banegas

booleano Pila.Vacia()

inicio

retorna lo vacia

Fin

Tipo elemento Pila.cima()

inicio

1. recupera (1. primero, E)

retornar E

Fin

## 2.1.4.2 Implementación con Vectores

Dirección del tipo Entero

Clase Pila

Atributos

elementos [max] vector de tipo Elemento

tope de tipo Dirección

Metodos

Crear()

booleano Vacia()

Meter (Es Tipo Elemento)

Sacar (Es E ? Tipo Elemento)

Tipo Elemento cima()

Fin

Pila. Crear

inicio

tope = 0

Fin

booleano Pila.Vacia()

inicio

retornar (tope = 0)

Fin

Pila. sacar (Es E ? Tipo Elemento)

inicio

Si no vacia() entonces

d = elemento [tope]

tope = tope - 1

Caso contrario // Error

Fin



Materia:

Fecha:

2.1.4.3

Implementación con simulación de memoria

Diego Fernando Barajas

Nodo

elemento TipoElemento  
sig Puntero a Nodo  
// Fin definición

Dirección Puntero a espacio de memoria de tipo Nodo

Clase pila

Atributos

tope Puntero de tipo Dirección

Métodos

crear()

booleano vacía()

meter (E: TipoElemento)

Sacar (Es E: TipoElemento)

TipoElemento cima()

Fin

Constructor crear

inicio

tope = -1

Fin

Pila.meter (E: TipoElemento)

inicio

aux = // Pedir espacio de memoria para nodo

Si aux < 0 no se pudo entonces

Ponerdato (aux, '-> elemento', E)

Ponerdato (aux, '-> sig', tope)

tope = aux

Caso contrario // Error

Fin

TipoElemento Pila.cima()

inicio

Si (vacía()) entonces // Error

Caso contrario

return obtenerdato (tope, '-> elemento')

Fin



que pueden aplicar en el concepto de  
según (entien) se coloca en la fila  
do de acuerdo son visible en las  
del al punto un salto.  
dato!! El concepto anterior  
se desliza como una  
ven a ser mejor cada  
El concepto anterior  
esto es el

Diego Fernando Bargas

que pueden aplicar en el concepto de  
según (entien) se coloca en la fila  
do de acuerdo son visible en las  
del al punto un salto.  
dato!! El concepto anterior  
se desliza como una  
ven a ser mejor cada  
El concepto anterior  
esto es el

Diego Fernando Bargas

Materia: \_\_\_\_\_

Fecha: \_\_\_\_\_

Diego Fernando Bargas

Pila = sacar (Es E % Tipo Elemento)

Inicio

Si (vacía()) Entonces // Error

Caso contrario

X = tope

E = obtener dato (Tope, '→ elemento')

Tope = obtener dato (tope, '→ sig')

Delete = espacio (x)

Fin



Diego Fernando Braneas Bruso

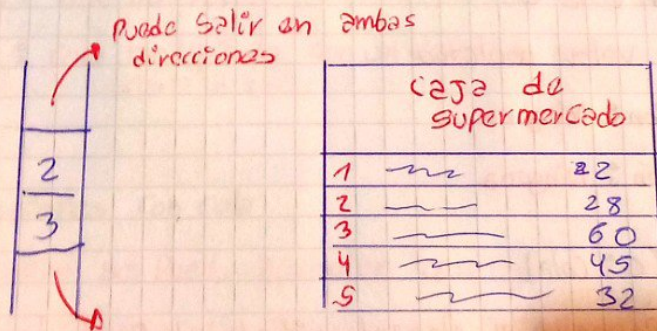
## UNIDAD VII TDA COLA

### 2.1.1 Descripción del TDA Cola

ahora conoceremos el concepto de cola y se definirá como "una estructura de almacenamiento donde los datos van a ser insertados por un extremo y serán extraídos por otro". El concepto anterior se puede simplificar como FIFO (first-in, first-out), esto es el primer elemento en entrar debe ser el primero en salir.

los ejemplos sobre este mecanismo de acceso, son visible en las filas de un banco los clientes llegan (entran) se coloca en la fila y esperan su turno para salir.

varios hechos de la vida que podemos aplicar en el concepto de colas son, por ejemplo, cuando las personas hacen fila para esperar al uso de un teléfono público, para subir al transporte escolar o un autobús, para realizar los pagos en un supermercado para imprimir varios documentos etc.





Nombre:

Fecha:

Diego Fernando Panegas Bruno

### 2.1.2 Especificación del TDA cola

Partiendo de lo establecido en la Unidad 1. Especificación in-

-Formal tenemos lo siguiente: elemento que conforma la estructura

TDA cola (valores: todos los valores numéricos de cualquier tipo,

OPERACIONES: crear, Vacía, Primero, Poner, Sacar)

#### OPERACIONES

Crear (C: cola)

Utilidad: Sirve para inicializar la cola

Entrada: Cola C

Salida: Valor booleano

Precondición: Ninguna

Poscondición: Ninguna

Vacía (C: cola)

Utilidad: Sirve para indicar si la cola está vacía

Entrada: Cola C

Salida: Valor booleano

Precondición: Ninguna

Poscondición: Ninguna

Primero (C: cola)

Utilidad: Devuelve el elemento situado al frente de la cola



Materia:

Fecha:

Entrada: C cola

Salida: Elemento

Precondición: La cola es no vacía

Poscondición: Ninguna

Poner (Es C: cola, E: Elemento)

Utilidad: Añade el elemento E a la cola, situado al final elemento

Entrada: C cola y Elemento E

Salida: Ninguna

Precondición: cola con capacidad

Poscondición: cola modificada en un elemento más en su contenido

Sacar (Es C: cola, Es E: Elemento)

Utilidad: Suprime el elemento situado en el frente de la cola y retorna

Entrada: Cola C y receptor de Elemento E

Salida: ninguna, el valor de elemento retorna en el parámetro

Por referencia E

Precondición: cola no vacía

Poscondición: cola modificada con un elemento menos en su contenido

### 2.1.3 Aplicaciones con cola

En esta sección se pueda apreciar que ya estamos en condición para plantear algoritmos usando el TDA cola, abstrayendo



Materia:

Fecha:

Diego Fernando Benegas

de la forma implementado.

concatenación de dos colas

Concatenar (cola cola1, cola cola2, cola cola3)

Inicio

~~Cola~~ mientras No (cola1.Vacia()) = verdadero)

Inicio

Cola1. Sacar (E)

ColaAux. Poner (E)

Fin

mientras no (ColaAux.Vacia())

Inicio

ColaAux. Sacar (E)

Cola3. Poner (E)

Cola1. Poner (E)

Fin

mientras no (Cola2.Vacia()) = verdadero)

Inicio

Cola2. Sacar (E)

ColaAux. Poner (E)

Fin

mientras no (ColaAux.Vacia())

Inicio

ColaAux. Sacar (E)

Cola3. Poner (E)

Cola2. Poner (E)

Fin

Fin

Determinar si una expresión es Palíndromo

Es palíndromo (cada cadena)

Inicio Para cada  $i=1$  hasta longitud de cadena

Letra = cad[i]

Poner (letra)

C. Poner (letra)

Fin

ok = verdadero

mientras (no C.Vacia) y (ok = verdadero)

Inicio

P. Sacar (temple)

C. Sacar (temple)

AMERICAN IRIS



Materia:

Fecha:

```
    Si elemPila < elemCola
    entonces OK = Falso
    Fin
    retornar OK
Fin
```

Diego Fernando Bonayán

## 2.1.4 Implementación del TDA Cola

En esta sección mostraremos tres implementaciones para el TDA cola:

- implementación con lista
- implementación con vectores
- implementación con simulación de memoria/puntero

### 2.1.4.1 Implementación con lista

clase cola

Atributos

$L \text{ : lista}$

Metodos

crear()

bookearVacía()

Poner (E : TipoElemento)

Sacar (E : TipoElemento)

TipoElemento Primero()

Fin

Constructor cola = crear()

inicio

// crear objeto L

Fin

cola = Poner (E : TipoElemento)

inicio

1. inserta (1. Primero(), E)

Fin

cola = sacar (E : TipoElemento)

inicio

1. recupera (1. Fin(), E)

1. suprime (1. Fin)

Fin



Matrón:

Fecha:

Diego Fernando Banegas

booleano cola.vacia()  
inicio  
retorna la vacia  
Fin

TipoElemento cola.Primerol()  
inicio  
lo recupera (l.Pin, E)  
retorna E  
Fin

## 2.1.4.2 Implementación con vector

Implementación con vector considerando un solo uso máximo

Dirección de tipo Entero

Clase cola

Atributos

V[Max] vector del tipo TipoElemento

ini

Fin de tipo Dirección

Metodos

Crear()

booleano vacio()

Poner(E : TipoElemento)

Sacar(E : TipoElemento)

TipoElemento Primerol()

Fin

cola.Crear()

ini

Fin=0

ini=1

Fin

booleano cola.vacio()

inicio

retornar (ini > Fin)

Fin

cola.Poner (E : TipoElemento)

inicio

Si  $Fin \leq Max$  entonces

$Fin = Fin + 1$

$V[Fin] = E$

Fin



Cola = sacar ( E : TipoElemento )

Inicio

Si no vacio() entonces

E = V[inicio]

inicio = inicio + 1

Fin

TipoElemento Cola = Primeros()

Inicio

Si no vacio() entonces

retorna V[inicio]

Fin

Implementación con vectores desplazado cada vez que se retira

Dirección de tipo entero

Clase cola

Atributos

V[Max] vector TipoElemento

ini

Fin de tipo dirección

Metodos

crear()

Poner ( E : TipoElemento )

sacar ( E : TipoElemento )

TipoElemento Primeros()

Fin

Cola = crear()

inicio

Fin = 0

ini = 1

Fin

booleano Cola = Vacio()

inicio

retornar (ini > Fin)

Fin

Cola = Poner ( E : TipoElemento )

Inicio

Si Fin < MAX entonces

Fin = Fin + 1

V[Fin] = E

Fin

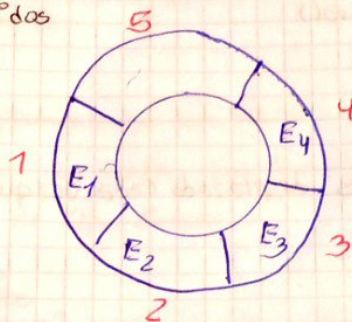


Tipo Elemento cola = Primerol()

Diego Fernando Barrera

inicio  
Si no vacio () entonces  
retorne VC1)  
fin

implementacion con vectores simulados que los extremos vector  
estan unidos



En esta forma de implementacion se usa un vector para guardar  
los elementos y se pierde una casilla que sirve como  
frontera entre el ultimo y primer elemento cola  
así mismo se puede apreciar que es necesario tener una  
funcion que retorne la direccion siguiente

Siguiente (dir  
inicio

Si dir = max entonces retornar 1  
caso contrario retornar dir + 1

fin

Direccion de tipo entero

Clase cola

Atributos

V[max] vector del tipo TipoElemento

ini

fin de tipo direccion



Materia:

Fecha:

Diego Fernando Banegas

Metodos  
Privados  
Direccion siguiente (dir)  
publicas  
crear ()  
booleano vacio ()  
Poner (E es Tipo Elemento)  
Sacar (E es Tipo Elemento)  
Tipo Elemento Primero ()  
Fin

Cola. crear  
inicio  
Fin = 0  
ini = 1  
Fin

booleano Cola vacio ()  
Inicio  
retornar (siguiente (Fin) = ini)  
Fin

Cola. Poner (E es Tipo Elemento)  
Inicio  
Si siguiente (siguiente (Fin)) < ini entonces  
Fin = siguiente (Fin)  
VER[Fin] = E  
Fin

Cola. Sacar (E es Tipo Elemento)  
Inicio  
Si no vacio () entonces  
E = VER[ini]  
ini = siguiente (ini)  
Fin

Tipo Elemento Cola. Primero ()  
Inicio  
Si no vacio () entonces  
retornar VER[ini]  
Fin



Materia:

Fecha:

2.1.4.3 implementación con  
simulación de memoria / puntero

Diego Fernando Panegas

Nodo

~~Crear~~ Elemento Tipo Elemento  
Sig puntero Nodo  
// Fin definición

Dirección puntero a espacio de memoria tipo de

clase cola

Atributos

ini, fin, puntero de tipo Dirección

metodos

Crear ()

booleano vacia ()

Poner (E: Tipo Elemento)

Sacar (E: Tipo Elemento)

Tipo Elemento Primer ()

Fin

Constructor crear

inicio

ini = -1

fin = -1

fin

booleano cola = vacia ()

inicio

retornar (ini = nulo)

Fin

cola = Poner (E: Tipo Elemento)

inicio

aux

si aux <> nulo entonces

Poner dato (aux, '→ elemento', E)

Poner dato (aux, '→ sig', nulo)

si vacia () entonces

ini = aux

fin = aux

caso contrario

poner dato (fin, '→ sig', aux)

fin = aux

caso contrario // Error

Fin



Materia:

Fecha:

Tipo Elemento ~~Sacar~~ Cola (N:índice)

Diego Fernando Boncyas

Inicio

Si (Vaca) entonces // Error

(caso contrario)

return obtener dato (ini' - elemento)

Fin

Cola = Sacar (E: Tipo Elemento)

Inicio

Si no vacia entonces

E = obtener dato (ini' - elemento')

X = ini

ini = obtener dato (ini' - sig')

Delate = espacio(x)

Fin



Diego Fernando Bonegas Bruno

Entrada : Pila P

Salida : Elemento

Precondición : La Pila es no vacía

Poscondición : Ninguna

Meter (Es P : Pila, E : Elemento)

Utilidad : Sup Añade el elemento E a la Pila, quedando situado en el tope

Entrada : Pila P y Elemento E

Salida : Ninguna

Precondición : Pila con capacidad

Poscondición : Pila modificada con un elemento más en su contenido

Sacar (Es P : Pila, Es E : Elemento)

Utilidad : Suprime el elemento situado en el tope de la Pila y lo Retorna

Entrada : Pila P y receptor de elemento E

Salida : Ninguna, el valor del elemento retorna en el Parámetro Por Referencia E

Precondición : Pila no vacía

Poscondición : Pila modificada con un elemento menos en su contenido



## 2.1.2 Especificación del TDA Pila

Partiendo de lo establecido en la unidad 1. Especificación informal tenemos los siguientes elementos que conforman la estructura

TDA Pila (VALORES todos los valores numéricos de cualquier tipo, Operaciones crear, vacía, cima, meter, sacar)

Operaciones  $\Leftrightarrow$  OPERACIONES

Crear (P: Pila)

Utilidad: Sirve para inicializar la Pila

Entrada: Pila P

Salida: Valor booleano ninguna

Precondición: ninguna

Poscondición: Pila inicializada

Vacía (P: Pila)

Utilidad: Sirve para identificar si la pila está vacía

Entrada: Pila P

Salida: Valor booleano

Precondición: Ninguna

Poscondición: Ninguna

Cima (P: Pila)

Utilidad: Devuelve el elemento situado en la cima de la Pila